Part 1: Your own API

This is one lab in four parts, and the same API grows all the way. Part 4 is a bonus. Here you build a **movies** API (id, title, director, year) with a Python list as data. Write it yourself, step by step, test each step at `/docs`, and read error messages from the bottom up.

Endpoints you build

```
GET     /movies
GET     /movies/{movie_id}
POST    /movies
PUT     /movies/{movie_id}
DELETE  /movies/{movie_id}
```

Steps

1 Start the server
Create a folder `movies_api` with a file `main.py`. Create the app, add a list of four movies (id, title, director, year), and write the route `GET /movies` that returns the list. Start it with `fastapi dev main.py` and open `/docs`.

2 Get one movie
Add `GET /movies/{movie_id}`. If the id does not exist, answer with `HTTPException(status_code=404)`. Test an id that exists, one that does not, and the text `abc`. Which status codes do you get?

3 Filter with a query parameter
Let `GET /movies` take an optional `director` and an optional `year`. Without any of them you get all movies. Try `/movies?director=scott` and `/movies?year=1999`.

4 Add a movie
Write a `Movie` class with Pydantic (title, director, year). Add `POST /movies` with `status_code=201` that gives the new movie the next free id, adds it to the list and returns it. Test it in `/docs`. Then send a movie without a director and a year written as text. What do you get, and where does it tell you what is wrong?

5 Replace and remove
You will now look up a movie by id three times, so move that code into a function `find_movie(movie_id)` that returns the movie or raises the 404. Use it in `PUT /movies/{movie_id}` (takes a full `Movie`) and in `DELETE /movies/{movie_id}` (answers 204). Delete the same movie twice. What happens the second time? Then save the file so that the server restarts, and check what happened to the movies you added.

6 Call your own API from Python
Copy `api_client.py` from yesterday into the folder. In a new file `client.py`, use `get_json` to get all movies and one movie that does not exist, and use `requests.post` to add a movie. Run the server in one terminal and `client.py` in another. Print the status code of the POST. Expected: the list of movies, The `server said no: 404` for the missing movie, and 201 for the POST.

Part 2: PATCH

In the lecture I skipped PATCH on purpose. PUT replaces the whole movie, so the client has to send every field. PATCH only changes the fields that are sent, and everything else stays as it was. To build it you need a second model where every field is optional.

The idea

```
class MovieUpdate(BaseModel):
    title: str | None = None
    director: str | None = None
    year: int | None = None

changes.model_dump(exclude_unset=True)   # only the fields the client sent
```

Steps

1 **Write the model and the route**
Add the class `MovieUpdate` above, and a route `PATCH /movies/{movie_id}` that takes the id and a `MovieUpdate`. Use `find_movie`, then update the movie with the fields the client sent. Hint: `movie.update(changes.model_dump(exclude_unset=True))`. Return the updated movie.

2 **Test that only one field changes**
In `/docs`, send a body that only contains `{"year": 2000}`. Check with `GET /movies/1` that the title and the director are unchanged.

3 **Test the edges**
What happens with an empty body `{}`? And with a year written as text? And with an id that does not exist? Write down the status code for each.

4 **Explain the difference**
Write two sentences, as a comment in your code, about when you would use PUT and when you would use PATCH.

Expected for step 2: only the year is changed. For step 3: an empty body changes nothing, text as year gives 422, and a missing id gives 404.

LEXICON

Part 3: Protect your API with a key

Right now anyone can add and delete movies. In this part you protect the routes that change data with an **API key**. The client sends the key in a header, like the Bearer token yesterday, and your API refuses everybody who does not have it. Reading movies stays open for everyone.

The idea

```
def check_key(x_api_key: str | None = Header(default=None)):
    if x_api_key != API_KEY:
        raise HTTPException(status_code=401)

@app.post("/movies", dependencies=[Depends(check_key)])
```

Steps

1 **Create the key and keep it safe**
Create `.gitignore` **first** and put `.env` in it. Then create `.env` and write `API_KEY=` followed by a long random text. You can make one with `python -c "import secrets; print(secrets.token_hex(16))"`. No quotes. Do not paste the key in chat or share your screen while it is visible.

2 **Read the key in your API**
Load it with `load_dotenv()` and `os.getenv("API_KEY")` at the top of `main.py`. Never write the key itself in the code.

3 **Write the check**
Write the function `check_key` from the idea above. Import `Header` and `Depends` from `fastapi`. FastAPI turns `x_api_key` into the header `X-API-Key` for you. Think about this: what happens if `API_KEY` is missing in `.env`? Make sure the API then refuses everybody, not accepts everybody.

4 **Protect the routes that change data**
Add `dependencies=[Depends(check_key)]` to POST, PUT, PATCH and DELETE. Leave the GET routes open.

5 **Test it**
In `/docs`, try POST without a key. You should get 401. Then add the key in the header field and try again. Then try a key with one wrong character. Which status code do you get each time?

6 **Use the key from Python**
In `client.py`, read the key from `.env` and send it with `headers={"X-API-Key": key}`. Try a POST with and without the key.

7 **Check that the secret is safe**
Run `git init` and then `git status`. The file `.env` must **not** appear in the list.

Expected for step 5: no key and a wrong key give 401, the right key gives 201. Why 401 and not 403? Because we do not know who the client is.

LEXICON

Part 4: Bonus

Your movies disappear when the server restarts, because they only live in memory. In this part you save them in a JSON file, so that they survive. This is the same idea as the cache in yesterday's bonus, but now the file is your database.

The idea

```
when the server starts:
    if movies.json exists: read the movies from it
    else: use the four starting movies

after every change (POST, PUT, PATCH, DELETE):
    write all movies to movies.json
```

Steps

1. **Write two helper functions**
   `load_movies()` reads the file if it exists and otherwise returns your starting list. `save_movies()` writes the whole list to the file with `json.dump`. Use `movies = load_movies()` instead of the hard-coded list.

2. **Save after every change**
   Call `save_movies()` in the routes that change data, just before they return.

3. **Test it**
   Add a movie, restart the server by saving `main.py`, and check that the movie is still there. Open `movies.json` and look at it.

4. **Think**
   What would go wrong if two people added a movie at exactly the same time? Write two sentences about it. Why do real applications use a database?

Useful tools

```
import json, os

os.path.exists(path)
json.load(file)
json.dump(data, file, indent=2)
```

Stuck? Read the error message from the bottom up, check `/docs`, print the data you are working with, and then ask.